DSAN 6700 Homework 3: User Feedback

Team members: Yuhua Pang, Estella Zeng, Ken Qin, Norman Wang

Our application (Entity Resolver): http://149.165.175.14:8000/

Our GitHub repository: https://github.com/normangw/entity-resolver

User brief: https://github.com/normangw/entity-resolver/blob/main/USER_BRIEF.md

Testing window: November 1–11, 2026

Application we tested (Operation: Negative One): http://149.165.154.219:8080/ui/

Repository we tested: https://github.com/dt7860920/6700-team-project

Summary of our testing

Norman tested Operation: Negative One, a vulnerability-scanning agent, through its API and web UI as a security analyst would. We exercised the allowlist targets (PyGoat and others), pasted-code snippets, malformed requests, and several scans started at the same time. We filed 10 findings as separate GitHub issues on their repository: 5 bugs or observations and 5 design questions.

Bugs and observations using API calls

1. A snippet scan missed an obvious command injection (CWE-78) that the static-analysis stage had flagged.
2. A snippet scan with no code was accepted (202) instead of rejected.
3. Unknown-target errors were inconsistent: 403 for nonexistent but a 422 pattern error for a URL.
4. Three scans started back to back were all accepted, and the README promises a 429 when busy.
5. Every finding reported confidence: 1.0 even though none were verified.

Design questions

6. How can a user tell "no vulnerability" apart from "the model missed it"?

7. What happens to a scan and its results after the page closes or the server restarts?

8. PyGoat is the golden-path SQL injection target, but a scan found only unsafe deserialization from 3 candidates. How complete should users assume a scan is?

9. Why is snippet Python-only, and what happens with pasted JavaScript?

10. Where is USER_BRIEF.md? Testers have no stated user, test steps or feedback request.

Yuhua tested Operation: Negative One at http://149.165.154.219:8080/ui/ on the service backend. I scanned Juice Shop, vulpy, the built-in Paste code sample, and a JavaScript snippet, then opened /info and looked up a scan id. Ten findings: 5 bugs or observations and 5 design questions.

Bugs and observations

1. Juice Shop, an allowlisted practice app, finished in 8.3s with “No vulnerabilities found.” Static analysis reported 0 candidates, and the model stage took 0 ms. The JSON message is “0 verified findings.”
2. The built-in Paste code sample’s SQL line was dismissed, but the dismiss reason says name is embedded in the SQL query with no sanitization. The page still says “No vulnerabilities found.”
3. That sample’s ping(host) line, which passes host to a shell, was dismissed as “input is not attacker-controlled.”
4. A pasted JavaScript SQL concatenation finished in 4.2s with 0 candidates and “No vulnerabilities found.” The model never read it.
5. The vulpy scan’s JSON says “3 verified findings” and the verify stage says “3 passed the verifier” in 0 ms. Every card says “not verified,” and the summary says “none verified yet.”

Design questions

6. Rung 2 is labeled “CWE + patch,” but vulpy’s rung 2 cards say “No patch proposed yet” and patch_diff is empty. The rung 1 card still shows CWE-89, which the legend says rung 1 does not have. Which label should a user trust?

7. Looking up scn_629d95b57ac8 required no login. Who is allowed to read another person’s scan?
8. The finished cards cannot be downloaded. How should an analyst keep a report?
9. /info on the public Jetstream URL reports "environment": "local" while model_backend is service. Is this site a local demo or the real deployment?
10. The page header says the tool proposes a fix. vulpy’s cards describe the SQL injection and then say the fixer is not enabled. Should the header describe what the service does today?

Ken tested Operation: Negative One at http://149.165.154.219:8080/ through its API, scripting each scan from a terminal the way an analyst would automate it. I focused on the scan options (max_findings, min_rung), finding lookups, input limits, and a pasted Flask snippet with seven planted flaws, and I scanned vulpy three times and PyGoat once. Ten findings: 5 bugs or observations and 5 design questions. On the positive side, the same snippet scanned twice gave identical results, a safe parameterized-query snippet produced no false positive, and only port 8080 answered from outside (8081, 6333 and 11434 were closed).

Bugs and observations

1. Finding ids are reused across scans. vulpy scan scn_aca2939e3cc9 lists fnd_d5e3800d82, but after a second vulpy scan (scn_b1a0b8f7bc96), GET /findings/fnd_d5e3800d82 returns the newer scan's record, so the first scan's finding can no longer be opened by its own id.
2. A pasted Flask snippet with seven planted flaws (eval, pickle.loads, yaml.load, a path-joined send_file, a shell=True command, a hard-coded API key, and MD5 password hashing) produced only 3 SAST candidates. Path traversal, command injection, the secret and MD5 never reached the model, and yaml.load was then dismissed, leaving 2 findings.
3. With max_findings set to 1, vulpy returned its one rung 1 SQL injection and dropped the two rung 2 findings from the same scan. The capped scan still took 40s, the same as an uncapped one.
4. With min_rung set to 3, vulpy ran the full 28s pipeline and reported “0 verified findings.” Nothing says the 3 SQL injections were filtered out, so the result reads like a clean repository.
5. A snippet over 20,000 characters is rejected with a 422, but the error body echoes the entire pasted code back in its “input” field. The 20,000-character limit is not stated in the UI or the README.

Design questions

1. Where does pasted code go? Each scan creates a workspace (.ono-workspace/scn_…) and sends code to a qwen2.5-coder:7b endpoint. Is the code kept, logged or deleted after the scan, and does the model run on your instance or somewhere else?
2. How would an analyst scan their own code? The allowlist is three public practice apps plus a Python paste box, with no file or repository upload. Who can add a new target, and how?
3. How is severity decided? eval was rated “critical” and pickle “high.” Does the model choose it, or is it mapped from the CWE? Analysts triage by severity, so it needs to be consistent.
4. The API accepts max_findings and min_rung, but the UI offers neither, and a PyGoat scan sent with options.snippet was accepted (202) with the snippet silently ignored. Are these options meant for users, and should conflicting options be rejected?
5. max_candidates caps a scan at 20 SAST candidates, and the UI says scans “take minutes,” while ours took 28–42s on small targets. A real repository can have hundreds of Semgrep hits. How will candidates be ranked, and what runtime should a user expect?

Hongjin tested Operation: Negative One at http://149.165.154.219:8080/ui/ through the web interface using OWASP PyGoat, vulpy, and the built-in pasted-code sample. The testing focused on starting scans, understanding progress, and responding to failures. All three scans encountered the same model-endpoint connection error. The observations below describe this shared failure and its effects, rather than five separate bugs.

Bugs and observations

1. PyGoat scan failed during agent analysis. Scan `scn_b25a031f9022` failed after 1.8 seconds. Target fetching succeeded, and static analysis identified three candidates, but the model-analysis stage could not proceed.
2. vulpy scan encountered the same error. Scan `scn_6961eb0ee6a9` failed after 2.4 seconds. Static analysis identified three candidates, but agent analysis failed because the model endpoint was unreachable.
3. The built-in pasted-code sample also failed. Scan `scn_3e55b578ad1d` failed after 0.5 seconds. Static analysis identified two candidates, labeled CWE-89 and CWE-78, but model analysis did not complete.
4. The application accepted scans before reporting an unavailable model endpoint. All three scans started on the `service` backend and then returned: `model endpoint unreachable at` http://localhost:11434/v1`: [Errno 111] Connection refused`. Verification was not reached. The screenshots establish the connection failure but do not establish its underlying cause.
5. The failure message was clear, but the recovery instructions were intended for administrators. The interface correctly displayed `FAILED` and explained that the scan had not completed. However, it instructed users to check server logs, `MODEL_SERVICE_URL`,

`EXECUTOR_URL`, and `make executor`. A tester accessing the public application cannot perform these server-side actions.

Design questions

1. Should service readiness be checked before a scan starts? If the model endpoint is unavailable, could the interface display a temporary-unavailability message before users submit a scan?
2. What should an ordinary user do after this failure? Could the error message provide user-facing guidance, such as when to retry or whom to contact, separately from administrator troubleshooting instructions?
3. Should partial results remain available? Static analysis identified candidates before each failure. Could users inspect these candidates with clear labels indicating that they have not undergone model analysis or verification?
4. How should retrying a failed scan work? After the model service is restored, should users resume the existing scan or create a new one? Would a retry repeat all stages or only the failed stage?
5. Could the interface simplify reporting failures? A copyable error summary containing the scan ID, target, failed stage, and timestamp would help users provide useful feedback without manually transcribing technical details.